

RAFAIL: Relationship-Aware Failure Detection for Robotic Manipulation

Loris Schneider*, Edgar Welte*, Rania Rayyes

Abstract—Detecting failures during execution is essential for reliable robotic manipulation. Vision-language models (VLMs) can assess task outcomes semantically but add runtime computation, whereas out-of-distribution (OOD) detectors may respond to harmless scene variations rather than failure-relevant deviations. We introduce RAFAIL, a framework for detecting execution failures during robotic manipulation. RAFAIL identifies failures by detecting anomalies in task-relevant relationships between entities, such as a gripper and an object or an object and its target. By focusing OOD detection on relevant parts of the observation, RAFAIL reduces sensitivity to task-irrelevant scene variation. Offline, a VLM annotates successful demonstrations with task progress and relationship importance, which are used to learn point-cloud-based relationship representations without relying on policy-internal features. At runtime, relationship-specific OOD detectors evaluate these representations while relationship importance and task progress are predicted without VLM inference. RAFAIL requires no failure data and achieves 73.4% balanced accuracy across three real-world robotic manipulation tasks, outperforming the strongest evaluated OOD- and uncertainty-based baselines.

I. INTRODUCTION

Robot learning has enabled increasingly complex manipulation tasks [1], [2], yet learned policies remain prone to failures during real-world execution due to uncertainty in the environment, perception, and unexpected physical interaction. Detecting such failures is essential for reliable autonomous operation, allowing execution to be stopped or recovery to be initiated before failures cause damage [3]–[5]. The central challenge is not merely detecting that an execution differs from previously successful behavior, but determining whether that deviation affects task success.

Detectors leveraging visual language models (VLMs) use visual observations and task descriptions to assess whether execution satisfies a goal [4], [6], [7]. Although they can recognize semantic failures, continuous VLM inference adds substantial runtime cost [5], [8]. Out-of-distribution (OOD) and uncertainty-based detectors can instead be trained on successful executions and evaluated efficiently during deployment [3], [9]. However, their signals indicate deviation or uncertainty rather than failure: a harmless change in the scene can trigger an alert, while a subtle but consequential interaction error may be missed [8]. Many such detectors additionally rely on policy-internal features that may not be easily available across policy architectures [5].

*Equal contribution.

Institute for Material Handling and Logistics (IFL), Karlsruhe Institute of Technology, 76131, Germany. Email: loris.schneider@kit.edu. This work is supported by the German Federal Ministry of Research, Technology, and Space (BMFTR) under the Robotics Institute Germany (RIG), the DFG SFB-1574-471687386 project, and the Ministry of Science, Research and Arts of the Federal State of Baden-Württemberg within the InnovationCampus Future Mobility.

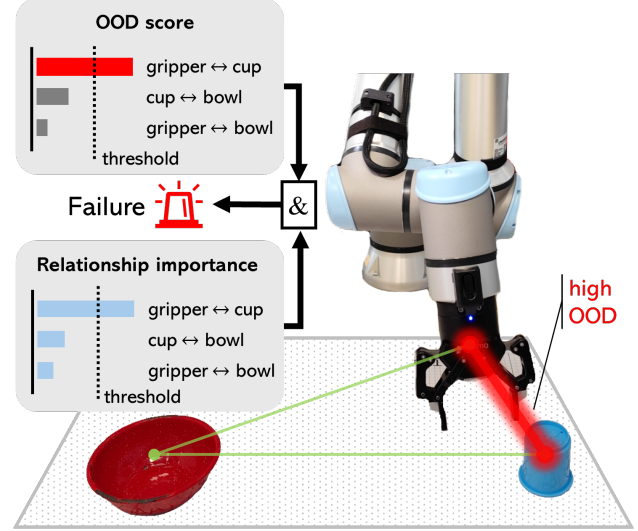


Fig. 1. RAFAIL detects failures by monitoring task-relevant relationships. Relationship importance, estimated from successful demonstrations, gates the contribution of each relationship at runtime. A failure is triggered when an important relationship exhibits a high OOD score, while deviations in less relevant relations are suppressed.

We address these challenges by focusing failure detection on task-relevant relationships between entities rather than global observations or policy representations. In manipulation, task success often depends on how these relationships evolve, such as between a gripper and an object or an object and its target. We therefore introduce relationship-aware failure detection (RAFAIL), a framework for robustly detecting execution failures during robotic manipulation. RAFAIL represents individual relationships from point-cloud observations and applies task-progress-conditioned and importance-gated OOD detection to each. A VLM provides semantic supervision only offline by annotating successful demonstrations with task progress and relationship importance, while runtime detection requires neither VLM inference nor access to policy-internal features. Relationship-specific OOD scores additionally indicate which interaction deviates from successful execution. Our contributions are:

- A **relationship-aware framework for failure detection** that decomposes global observations into dedicated representations of individual task-relevant relationships.
- A **relationship importance and task progress estimation approach for failure detection** via distillation of semantic task understanding from a VLM, enabling relationship importance and task progress estimates without online VLM inference.
- A **task-progress-conditioned relationship OOD detector** that localizes which interaction is anomalous, in-

tegrates task progress and relationship importance, and is independent of policy-internal latent representations. RAFAIL is evaluated on three real-world robotic manipulation tasks, achieving a balanced accuracy of 73.4% versus 65.2% for the best evaluated baseline.

II. RELATED WORK

A. Foundation Models for Failure Detection

The semantic understanding of foundation models, especially VLMs, makes them attractive for application in failure detection as they can derive task understanding and task success conditions. AHA [6] finetunes a VLM on synthetic manipulation failures for detection and explanation, while FailSafe [4] additionally learns recovery behavior. I-FailSense [7] uses VLM features for semantic failure classification. Their framework outperforms larger non-finetuned VLMs, as well as AHA [6], in failure detection tasks. Those works show that adapted foundation models, especially VLMs, work well for failure detection and reasoning, especially for the detection of semantic failures. But continuously running VLM inference alongside a policy creates a large computational overhead [5], [8]. This is additional to the high computational demands of finetuning VLMs on large failure datasets. In contrast, RAFAIL does not rely on a VLM for online inference.

B. Uncertainty- and OOD-based Failure Detection

A second branch of failure detection methods is based on the intuition that policies are likely to fail in OOD situations and therefore aims to detect such situations rather than directly detecting failure. The general approach was described in [9] and called Fail-Detect. It consists of extracting signals from policy inputs and outputs during successful task demonstrations and training OOD-detectors on them. Then, they calculate Conformal Prediction (CP) [10] thresholds for those OOD scores. They evaluate the performance of diverse OOD-detectors for robot manipulation tasks. In real-world experiments, they identify their own logpZO [9] and Random Network Distillation (RND) [11] as the best performing learned detectors but also compare to non-learned methods like PCA-kmeans [9], [12]. While [9] evaluates OOD-detectors on observations, predicted actions, or both, [3] propose to also estimate the uncertainty of the action prediction. Their framework named FIPER uses RND as an OOD-detector on the observations. Additionally, FIPER samples batches of action chunks from the policy at each timestep and estimates their entropy as an uncertainty measure which they call Action Chunk Entropy (ACE). A failure label is only produced if both the RND OOD-detector and the uncertainty estimator exceed their respective CP thresholds. They show consistently high failure detection accuracies with this combined approach, outperforming pure OOD-detection or uncertainty estimation on most tasks.

Uncertainty- and OOD-based failure detection approaches are generally more computationally efficient and can run alongside a policy with little overhead. Crucially, they do not require additional failure data, using only successful

trajectories. Finally, they allow failures to be predicted before they occur. On the other hand, since those approaches detect deviations from successful inputs and/or outputs instead of actual failures, they are usually not able to distinguish failures from harmless deviations and might struggle to detect subtle failures [8].

The authors of [8] correctly identified this limitation and proposed to deploy a VLM as an additional semantic filter. If an OOD-detector passes its CP threshold, a heatmap is created that marks the location of the deviation in an RGB image. This image is input into a VLM which decides if the deviation is harmless or a failure. Since they use an off-the-shelf, non-finetuned VLM and query it only in OOD situations, they do not require failure data and reduce runtime computation demand, but still require full VLM inference during runtime.

In contrast to prior OOD- and uncertainty-based approaches that operate on global observations, actions, or policy representations, RAFAIL performs OOD detection on task-relevant object relationships. Semantic information from a VLM is used only offline to identify and weight these relationships, while runtime detection remains lightweight and independent of policy-internal features.

III. METHOD

A. Problem Formulation

a) *Imitation Learning Policy*: A 6-DoF robotic arm equipped with a parallel-jaw gripper operates over discrete control time steps $t \in \{0, \dots, T\}$. At each time step t , the robot receives an observation

$$\mathbf{o}_t = \left(\mathbf{o}_t^{\text{pcd}}, \mathbf{o}_t^{\text{prio}} \right),$$

consisting of a workspace point cloud $\mathbf{o}_t^{\text{pcd}} \in \mathbb{R}^{N^{\text{pts}} \times 3}$ and the robot's proprioceptive state $\mathbf{o}_t^{\text{prio}} \in \text{SE}(3) \times \{0, 1\}$. The robot executes an end-effector command $\mathbf{a}_t = (\mathbf{T}_t, g_t)$, where $\mathbf{T}_t \in \text{SE}(3)$ specifies the desired end-effector pose relative to the robot base, and $g_t \in \{0, 1\}$ denotes the binary gripper state (open or closed). The commanded pose is tracked by a low-level Cartesian impedance controller. Given the most recent K observations, a policy π predicts a sequence of H future actions according to

$$\hat{\mathbf{a}}_{t:t+H-1} = \pi(\mathbf{o}_{t-K+1:t}).$$

In addition to action prediction, the policy exposes auxiliary estimates of task progress and object relationship importance, which the failure predictor uses. Figure 2 shows the pipeline overview during online deployment.

b) *Failure Predictor*: We employ an independent failure predictor that estimates whether the current execution is likely to fail. At each time step t , the predictor receives the current scene RGB and depth image $\mathbf{o}_t^{\text{img}}, \mathbf{o}_t^{\text{depth}}$ together with auxiliary semantic information $\mathbf{c}_t = (\mathbf{r}_t, p_t)$, where $\mathbf{r}_t$ denotes the estimated importance of the relationships and p_t represents the current task progress. Based on these inputs, the failure predictor outputs a failure signal f_t^{fail} , indicating the possibility of an execution failure at time step t .

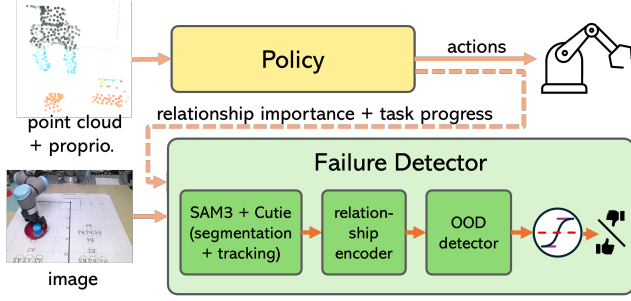


Fig. 2. Overview of online deployment. The policy predicts actions based on point cloud and proprioceptive observations. It also predicts auxiliary scores for relationship importance and task progress. The failure detector module uses the image observation and the auxiliary scores to segment and score each relationship via an OOD detector.

B. Method Overview

Figure 3 summarizes the offline training pipeline. During training, the VLM-generated pairwise preferences supervise the importance and progress labelers. These labelers are used only offline to annotate every successful demonstration. Their frame-wise outputs then serve as supervised targets for auxiliary heads attached to the manipulation policy. Consequently, neither the VLM nor the importance/progress labelers are executed at deployment; the policy directly predicts relationship importance $\hat{r}_t$ and task progress $\hat{p}_t$ jointly with its action output.

C. Object-Relationship Representation

We hypothesize that many task-level manipulation failures manifest as incorrect relationships between task-relevant entities. Entities refer to movable objects, stationary environment elements, and robot parts relevant to the current task. An object relationship describes the specific state of a pair of objects. We therefore focus only on object pairs to detect failures or abnormalities. We extract task-relevant objects from a scene and task description generated by a scene-understanding VLM. We provide a VLM [13] with frames from a task demonstration arranged in a single grid image and prompt it to describe the task, its start and goal states, and the task-relevant objects. We then refine the initial task description by providing grid images of different demonstrations of the same task, resulting in a more general description and alternative object descriptors. From the list of task-relevant objects, we create all unique object pairs and use those as the task-relevant object relationships.

D. Relationship Importance Estimation

The importance of an object relationship at a specific task stage must be inferred from semantic information while considering the task context. This abstract concept is not easily quantifiable. Therefore, we distill numeric relationship-importance labels from a VLM that can relate semantic information in an observation to the task context and goal. First, we provide a VLM [14] with the task description along with two grid images, each showing a set of $L = 4$ consecutive frames $X = (I_t, \dots, I_{t+L-1})$ with a random start frame I_t and taken from a random task demonstration. We prompt the VLM to rank the importance of all object

relationships between the two frame sequences given the task description and allowing for equal importance. Additionally, we also prompt the VLM to decide which of the two frame sequences corresponds to further task progress, also allowing for equal task progress. We collect a dataset $\mathcal{D} = \{X_i^{(0)}, X_i^{(1)}, y_i, y_i^{\text{prog}}\}_{i=1}^{N^{\text{cmp}}}$, where y_i denotes the ranking of the relationship importance and y_i^{prog} denotes the ranking of the task progress with

$$y_i^k = \begin{cases} -1, & \text{if relationship } k \text{ is more important in } X_i^{(0)} \\ 0, & \text{if relationship } k \text{ is equally important in both} \\ +1, & \text{if relationship } k \text{ is more important in } X_i^{(1)} \end{cases}$$

and

$$y_i^{\text{prog}} = \begin{cases} -1, & \text{if } X_i^{(1)} \text{ is chronologically before } X_i^{(0)} \\ 0, & \text{if both correspond to the same task progress} \\ +1, & \text{if } X_i^{(0)} \text{ is chronologically before } X_i^{(1)}. \end{cases}$$

We use this dataset to train an importance labeler $\text{Imp}(X_i) = \mathbf{r}_i$ which consists of a ResNet18 [15] and an MLP head with a Sigmoid output layer, thus creating an R -dimensional score vector where each entry $r_i^k \in [0, 1]$ corresponds to the importance score for the relationship index $k \in \{1, \dots, R\}$. We train as follows: For each relationship index k in a comparison pair i , let $r_i^{k,(0)}, r_i^{k,(1)}$ denote the predicted scores for each frame sequence. We define the score difference as $\delta_i^k = r_i^{k,(1)} - r_i^{k,(0)}$. The strict-preference and tie sets are

$$\mathcal{S}_{\text{imp}} = \{i, k \mid y_i^k \neq 0\}, \quad \mathcal{M}_{\text{imp}} = \{i, k \mid y_i^k = 0\}.$$

and the strict preference loss is

$$\mathcal{L}_{\text{strict}} = \frac{1}{|\mathcal{S}_{\text{imp}}|} \sum_{i,k \in \mathcal{S}_{\text{imp}}} \log \left(1 + \exp \left(-\frac{y_i^k \delta_i^k}{\tau_{\text{imp}}} \right) \right),$$

with the temperature τ_{imp} as a hyperparameter. Additionally, we define a tie loss for comparisons in the tie set as

$$\mathcal{L}_{\text{tie}} = \frac{1}{|\mathcal{M}_{\text{imp}}|} \sum_{i,k \in \mathcal{M}_{\text{imp}}} [\max(0, |\delta_i^k| - m_{\text{tie}})]^2,$$

where the hyperparameter m_{tie} denotes the acceptable score difference margin. The total loss is $\mathcal{L} = \mathcal{L}_{\text{strict}} + \lambda_{\text{tie}} \mathcal{L}_{\text{tie}}$, with the weight hyperparameter λ_{tie} . After training, we score the relationship importances for each frame stack in each episode and assign the frame-wise labels via a moving average window.

E. Task Progress Estimation

In a similar fashion, we train a task progress labeler $\text{Prog}(I_{1:T}) = \mathbf{p}$, where $I_{1:T}$ is the complete frame sequence of task demonstration and $\mathbf{p}$ is a T -dimensional vector assigning each frame a task progress label. The task progress labeler consists of a ResNet18 that creates latent features $h_t = \text{ResNet18}(I_t)$ from each frame in a task demonstration. The sequence of frame features $h_{1:T}$ is fed into a BiLSTM [16] which creates temporal features $u_{1:T} =$

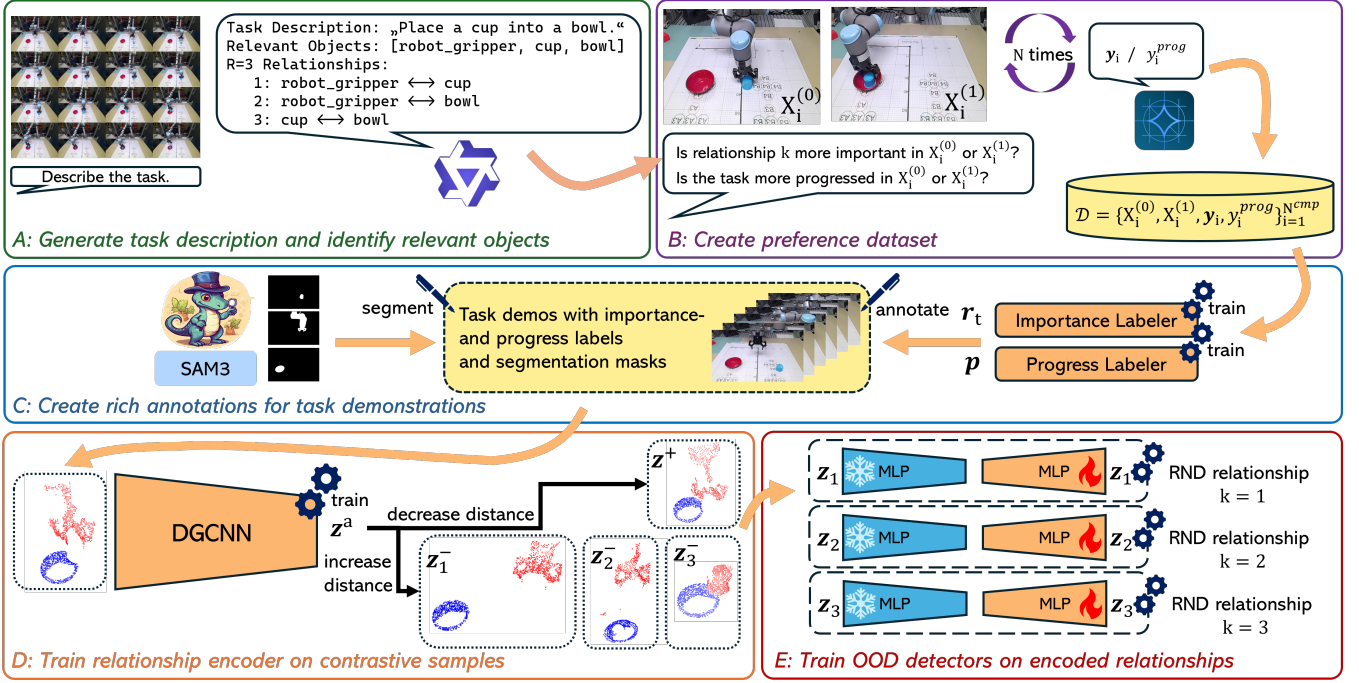


Fig. 3. Overview of the offline training process. (A) Initially, a task is presented to a VLM as a sequence of images to extract a detailed task description, including relevant object relationships. (B) In the second stage, a preference dataset is created by labeling the importance of relationships and task progress using a smaller VLM. (C) The preference dataset is distilled into two labeling networks to generate a rich dataset for subsequent encoder and policy training. (D) The relationship encoder is trained to create an enriched, relationship-aware latent representation. (E) Out-of-distribution (OOD) detectors for each relationship are trained exclusively on successful demonstration data.

BiLSTM($h_{1:T}$). Finally, an MLP head predicts positive task progress increments

$$\Delta_t = \log(1 + \text{softplus}(\text{MLP}(u_t))) + \epsilon, \quad t = 1, \dots, T-1.$$

We calculate a normalized temporal weight as

$$w_t = \frac{\Delta_t}{\max\left(\sum_{n=1}^{T-1} \Delta_n, \epsilon\right)}, \quad t = 1, \dots, T-1,$$

from which we calculate the frame-wise progress as:

$$p_1 = 0, \quad p_t = \sum_{n=1}^{t-1} w_n, \quad t = 2, \dots, T.$$

This calculation ensures continuously increasing task progress labels, ranging from 0 to 1, independent of the actual number of frames in a task demonstration. Since the monotonic construction already satisfies many strict progress comparisons, we filter the VLM comparison set to focus training on informative examples. The frame-wise progress predictions are first averaged over the same temporal windows of length $L = 4$ used for the VLM comparisons, yielding one predicted progress value $\hat{p}_i^{(j)}$ per sequence. We define the predicted progress difference as $\delta_i^{prog} = \hat{p}_i^{(0)} - \hat{p}_i^{(1)}$. We then filter equal-progress comparisons for consistency across demonstrations: comparisons whose temporal ordering conflicts strongly with other equal-progress comparisons are discarded, and only the most consistent fraction is retained. Since strict comparisons are often already satisfied by monotonicity, we preferentially retain those located temporally

close to the retained equal-progress comparisons and randomly sample additional strict comparisons until a predefined strict-to-equal ratio is reached. The resulting filtered equal- and strict-progress comparison sets are denoted by $\tilde{\mathcal{M}}_{prog}$ and $\tilde{\mathcal{S}}_{prog}$, respectively $\tilde{\mathcal{S}}_{prog} \subseteq \mathcal{S}_{prog}$, $\tilde{\mathcal{M}}_{prog} \subseteq \mathcal{M}_{prog}$ with

$$\mathcal{S}_{prog} = \{i \mid y_i^{prog} \neq 0\}, \quad \mathcal{M}_{prog} = \{i \mid y_i^{prog} = 0\}.$$

For the selected strict comparisons, we use a ranking loss

$$\mathcal{L}_{rank} = \frac{1}{|\tilde{\mathcal{S}}_{prog}|} \sum_{i \in \tilde{\mathcal{S}}_{prog}} \log \left(1 + \exp \left(\frac{y_i^{prog} \delta_i^{prog}}{\tau_{prog}} \right) \right),$$

where τ_{prog} is a temperature hyperparameter. For retained equal-progress comparisons, we penalize differences exceeding a margin m_{same} ,

$$\mathcal{L}_{same} = \frac{1}{|\tilde{\mathcal{M}}_{prog}|} \sum_{i \in \tilde{\mathcal{M}}_{prog}} [\max(0, |\delta_i^{prog}| - m_{same})]^2.$$

Additionally, $\mathcal{L}_{jump}$ quadratically penalizes progress increments w_t exceeding $\gamma/(T-1)$, discouraging the predicted progress from being concentrated in a small number of frames. The complete training objective is

$$\mathcal{L} = \lambda_{rank} \mathcal{L}_{rank} + \lambda_{same} \mathcal{L}_{same} + \lambda_{jump} \mathcal{L}_{jump}.$$

After training, the progress labeler is applied to each complete task demonstration to obtain a continuous frame-wise task progress label $p_t \in [0, 1]$, which can subsequently be used to identify corresponding task stages across demonstrations.

F. Relationship Representation Learning

We represent each relationship by encoding the point cloud of the respective object pair into a low-dimensional latent vector $\mathbf{z}$. The point clouds of the object pairs are obtained by detecting and segmenting the task relevant objects as defined by the task description in a single frame from a task demonstration recording using GroundedSAM2 [17]. Those initially segmented objects are then tracked through the whole task demonstration using SAM3 [18], producing fine-grained segmentation masks for each object.

The encoder is a DGCNN [19] which is trained using a contrastive loss directly on the latent vector. This loss is an importance-weighted ranking loss which compares the latent vectors of an anchor sample $\mathbf{z}^a$ to latent vectors of one positive example $\mathbf{z}^+$ and three negative examples $\mathbf{z}_j^-$, $j = 1, 2, 3$. The final loss function is

$$\mathcal{L}_{\text{rel}} = \frac{1}{3} \sum_{j=1}^3 \max(0, \alpha^+ d(\mathbf{z}^a, \mathbf{z}^+) - \alpha_j^- d(\mathbf{z}^a, \mathbf{z}_j^-) + m_{\text{rel}}),$$

with a distance measure d which we implement as the cosine distance $d(\mathbf{z}_1, \mathbf{z}_2) = 1 - \mathbf{z}_1^\top \mathbf{z}_2$ and a margin hyperparameter m_{rel} . The triplet terms are weighted using the estimated relationship importance scores of each sample

$$\alpha^+ = \max(r^a, r^+), \quad \alpha_j^- = \max(r^a, r_j^-),$$

with r^a, r^+, r_j^- denoting the importance score of the anchor, positive sample, and negative samples. Intuitively, the loss function encourages decreasing the distance of the anchor latent $\mathbf{z}^a$ to the positive sample $\mathbf{z}^+$ while increasing the distance to the negative samples $\mathbf{z}_j^-$, $j = 1, 2, 3$. We collect the positive and negative samples by comparing task progress label, demonstration, and relationship identity to the anchor sample. The positive sample shows the same relationship, is taken from a frame with the same task progress label, but from a different demonstration. The negative samples either show a different relationship from the same demonstration at the same task progress, the same relationship at a different task progress within the same demonstration, or the same relationship at a different task progress in a different demonstration. Table I provides a compact overview of the positive and negative samples. Instead of considering the full combinatorial set of possible negative samples, we restrict training to three representative cases that capture distinct types of relationship mismatch. These cases are chosen to provide comparatively difficult negatives, under the assumption that relationships involving different object pairs are generally easier to distinguish than different states of the same object pair.

G. Relationship-Aware Failure Prediction

To detect erroneous relationship states, we apply OOD detection to the learned relationship latent $\mathbf{z}$. As detailed in section II, several methods can be trained using only successful in-distribution samples [9], [11], [12]. We therefore do not introduce a new OOD detector. Instead, our relationship encoder provides a latent representation to which

Task Progress	Demonstration	Relationship	Target
same	different	same	positive
same	same	different	negative
different	same	same	negative
different	different	same	negative

TABLE I

OVERVIEW OF POSITIVE AND NEGATIVE SAMPLES FOR THE RELATIONSHIP ENCODER CONTRASTIVE LOSS.

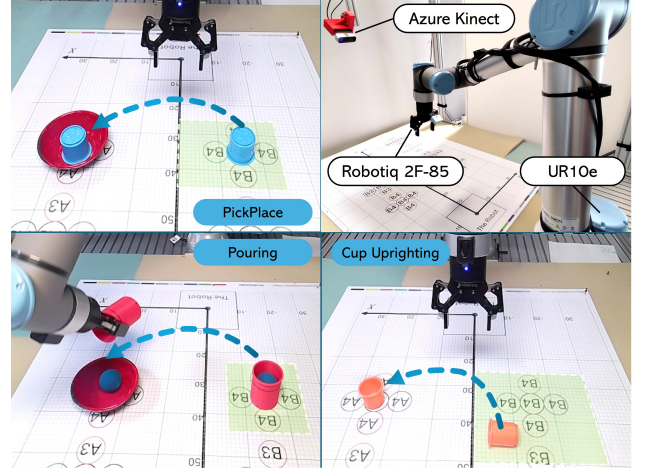


Fig. 4. Experimental setup and representative real-world tasks used in the experiments. Marked regions indicate the variance in task setup.

such detectors can be applied. While this makes our approach compatible with a broad range of OOD detectors, we choose RND [11] due to its fast inference and comparatively high performance. We use FiLM [20] to condition the RND OOD detector on the current task progress p_t .

For each relationship k , we train an independent detector ϕ_k on successful demonstrations and obtain an OOD score $s_t^k = \phi_k(\mathbf{z}_t^k, p_t)$. A relationship-specific threshold θ_{ood}^k is calibrated using CP [10]. To account for whether an anomalous relationship is relevant at the current task stage, we combine its OOD score with the predicted relationship importance $\hat{r}_t^k$. We use the importance as a gate,

$$f_t^k = \mathbb{I}[\hat{r}_t^k > \theta_{\text{imp}}] \mathbb{I}[s_t^k > \theta_{\text{ood}}^k], \quad f_t^{\text{fail}} = \max_k f_t^k.$$

The importance threshold is a hyperparameter which we set to $\theta_{\text{imp}} = 0.5$. An execution is classified as failed if $f_t^{\text{fail}} = 1$ at any timestep. During deployment, we use Cutie [21], initialized with GroundedSAM2 [17], for object segmentation as it offers improved real-time tracking capabilities compared to SAM3.

IV. EXPERIMENTS

A. Experimental Setup

a) *Tasks*: We evaluate our method on three different manipulation tasks as visualized in Figure 4: *PickPlace*, *Uprighting*, and *Pouring*, which require accurate grasping, insertion, and pouring interactions, respectively. In the *PickPlace* task, the robot needs to pick a cylindrical object and place it inside a small bowl. The *Uprighting* task requires the robot to lift and upright a fallen cup. In the *Pouring* task, the

robot needs to pick a cup and pour its content, a ball, into a small bowl.

b) Hardware and Data collection: We record demonstrations via teleoperation using a Meta Quest 3 VR headset on a UR10e robot arm with a Robotiq 2F-85 parallel jaw gripper, which is also used during experiments. We use an Cartesian impedance controller¹ to prevent a safety stop during physical contact. The image and point cloud are captured by an external Azure Kinect camera. Figure 4 visualizes the experimental setup. We collect 12 demonstrations for each task. The recordings consists of end-effector pose, gripper position, scene RGB/depth image, and point cloud, while the recorded action is the absolute end-effector pose and gripper position. Data is recorded at 10 Hz. The recorded successful teleoperated demonstrations are used only for training/calibration. None of the 178 autonomous policy executions used in Tables II-III are included in relationship encoder, OOD-detector, auxiliary-labeler training, or CP calibration.

c) Policy and Experiments: We train the robot manipulation policy ManiFlow [22], a flow matching policy for imitation learning, on the successful task demonstrations. The visual input is the scene point cloud. We extend the original architecture with two learnable output tokens: one for relationship-importance scores and one for task progress. Each token is mapped to its respective output dimension by a small MLP with a sigmoid output activation, and both heads are trained jointly with the policy using mean-squared-error losses. In contrast to the policies used by [3], [9], ManiFlow does not force the visual input through a global bottleneck, creating an immediately usable, low-dimensional latent observation vector for OOD detection. Instead, ManiFlow deliberately retains local information by encoding each point as a token, thus mapping the input point cloud directly to a high dimensional token space, which leads to increased performance [22]. Using ManiFlow, we record 60 task executions for the *Uprighting* and *Pouring* tasks, and 58 task executions for the *PickPlace* task. We induce OOD situations by variations of the placement of objects and by placing task unrelated objects in the background. This results in 42 failed and 18 successful executions for the *Uprighting* task, 48 failed and 12 successful executions for the *Pouring* task, and 45 failed and 13 successful executions for the *PickPlace* task.

B. Baselines

As baselines, we choose successful and common real-time capable OOD- and uncertainty-based failure detectors. We choose Action Chunk Entropy (ACE), introduced by [3], as the uncertainty-based baseline and also compare against the combination with RND [11] termed FIPER [3]. Our OOD-detector baselines are lopO [9], logpZO [9], PCA-kmeans [12], and RND [11]. For all of those we compare obtaining the necessary global observation latent vector by

mean-pooling or max-pooling across the observation tokens and calculate separate thresholds using CP [10].

C. Failure Detection Performance

Table II compares the failure detection performance of our approach to the baselines for each task and across all evaluated executions (marked "Overall"). We report the balanced accuracy (Acc.), the true positive rate (TPR), and the false positive rate (FPR) with

$$\text{Acc.} = \frac{1}{2}(\text{TPR} + 1 - \text{FPR}), \quad \text{TPR} = \frac{\text{TP}}{P}, \quad \text{FPR} = \frac{\text{FP}}{N},$$

where P is the number of failed task executions (positives), N is the number of successful task executions (negatives), TP is the number of failed task executions that were correctly detected as failures (true positives), and FP is the number of successful task executions that were wrongly detected as failures (false positives).

Across all task executions, RAFAIL achieves the highest Acc. of 73.4% by combining a comparatively high TPR of 58.5% with a relatively low FPR of 11.6%. The second best overall method is ACE [3] which achieves an Acc. of 65.2%, a lower TPR of 46.7% and a higher FPR of 16.3%. RAFAIL achieves the highest Acc. for the *Pouring* task, where it also achieves the highest TPR, and for the *Uprighting* task. For the *PickPlace* task, RND and logpZO with max-pooling achieve the highest Acc. of 72.8% and 68.0% while RAFAIL reports the third-highest Acc. of 66.7%. It can be observed that several baseline methods struggle to maintain a consistent trade-off between TPR and FPR across tasks. Depending on the task, some methods appear to become either overly sensitive, with high TPR and FPR, or overly conservative, with low FPR but also low TPR. This suggests sensitivity to task-specific failure signals and limited robustness of a single detection configuration across tasks. Variants that use max-pooling appear to be overly sensitive, while mean pooling appears to be somewhat more stable, although often at the cost of lower TPR. Likely, this is because of the smoothing effect of mean-pooling, while max-pooling emphasizes the strongest response, increasing sensitivity but also susceptibility to benign variations. ACE shows comparatively consistent behavior, while RAFAIL maintains the most favorable TPR/FPR trade-off across all tasks, resulting in consistently high balanced accuracy.

We manually annotate the timestep in which the failure occurs in each task execution and record the timestep when failure is detected for each detection method. This manually annotated failure timestep serves as a reference rather than an independent ground truth, as the exact timestep of a failure can be ambiguous and may be annotated slightly differently by another observer. We compute the normalized signed detection delay as the difference between the detection timestep, divided by the maximum total execution time of successful executions, with negative values indicating early detection and positive values indicating late detection. The violin plots in Figure 5 show smoothed distributions. RAFAIL tends to trigger closer to the annotated failure

¹https://github.com/edgarwelteKIT/ur_impedance_driver_ros2

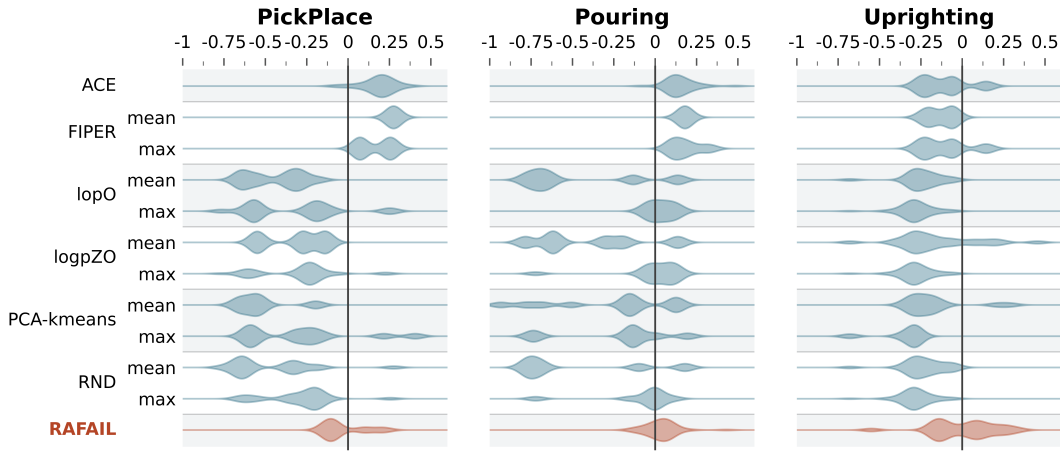


Fig. 5. Temporal alignment of failure detections with observed failures. For each failed execution, we manually identify the timestep at which the failure occurs and use it as the reference timestep, corresponding to the vertical line at zero. The horizontal axis shows the difference between the first detection timestep and this reference, normalized by the maximum duration of successful executions. Negative values indicate detections before the failure and positive values detections after it. Each violin plot shows the distribution of true-positive detection times for one method and task. The “mean” and “max” variants refer to mean- or max-pooling as a dimensionality reduction.

Task	Metric	ACE [3]		FIPER [3]		lopO [9]		logpZO [9]		PCA-kmeans [12]		RND [11]		RAFAIL (ours)
				mean	max	mean	max	mean	max	mean	max	mean	max	
PICKPLACE	Acc. (%) $\uparrow$	64.0	51.1	53.3	49.1	64.5	53.3	<u>68.0</u>	52.8	52.9	56.8	72.8	66.7	
	TPR (%) $\uparrow$	51.1	2.2	6.7	28.9	44.4	6.7	82.2	13.3	28.9	44.4	<u>53.3</u>	33.3	
	FPR (%) $\downarrow$	23.1	0.0	0.0	30.8	15.4	0.0	46.2	<u>7.7</u>	23.1	30.8	<u>7.7</u>	0.0	
POURING	Acc. (%) $\uparrow$	<u>71.9</u>	51.0	51.0	53.1	60.4	40.6	61.5	62.5	59.4	57.3	52.1	82.3	
	TPR (%) $\uparrow$	68.8	2.1	10.4	8.3	29.2	14.6	31.3	25.0	18.8	14.6	20.8	81.3	
	FPR (%) $\downarrow$	25.0	0.0	<u>8.3</u>	14.6	<u>8.3</u>	33.3	<u>8.3</u>	0.0	0.0	0.0	16.7	16.7	
UPRIGHTING	Acc. (%) $\uparrow$	55.7	54.8	54.4	61.1	50.0	63.1	50.0	<u>65.1</u>	47.2	64.7	50.0	71.4	
	TPR (%) $\uparrow$	16.7	9.5	14.3	50.0	100.0	<u>59.5</u>	100.0	<u>35.7</u>	16.7	57.1	100.0	<u>59.5</u>	
	FPR (%) $\downarrow$	<u>5.6</u>	0.0	<u>5.6</u>	27.8	100.0	33.3	100.0	<u>5.6</u>	22.2	27.8	100.0	16.7	
Overall	Acc. (%) $\uparrow$	<u>65.2</u>	52.2	52.9	53.6	53.8	51.3	55.7	59.9	52.6	58.4	53.7	73.4	
	TPR (%) $\uparrow$	46.7	4.4	10.4	30.4	56.3	26.0	69.6	24.4	21.5	37.8	56.3	<u>58.5</u>	
	FPR (%) $\downarrow$	16.3	0.0	<u>4.7</u>	23.3	48.8	23.3	58.1	<u>4.7</u>	16.3	20.9	48.8	11.6	

TABLE II

FAILURE DETECTION PERFORMANCE ACROSS REAL-WORLD ROBOTIC MANIPULATION TASKS. WE REPORT BALANCED ACCURACY (ACC.), TRUE POSITIVE RATE (TPR), AND FALSE POSITIVE RATE (FPR) IN PERCENT. THE “MEAN” AND “MAX” VARIANTS REFER TO MEAN- OR MAX-POOLING AS A DIMENSIONALITY REDUCTION. BEST RESULTS ARE MARKED IN BOLD AND SECOND BEST BY UNDERLINING.

timestep, especially for *PickPlace* and *Pouring*. Several baselines instead show distributions shifted substantially before or after the failure, suggesting that their OOD signals may respond to deviations that are less directly associated with the observed task failure. For *Uprighting*, RAFAIL shows a broader distribution. This may partly result from the nature of the task, where failures can develop gradually and may expose detectable relationship deviations at different times relative to the annotated failure timestep. Overall, the results in Figure 5 suggest that the OOD signals produced by RAFAIL are more closely aligned with the physical manifestation of failure than those produced by the baselines. This may partially explain RAFAIL’s more consistent TPR/FPR trade-off across tasks, as its detections tend to occur closer to the observed failure event.

To further assess whether the focus on individual relationships provides meaningful information about the failure, we manually annotate the relationships involved in each observed failure and compare them with the relationship

triggering the failure in RAFAIL. Among true-positive detections, the agreement between the relationship triggering the failure detection and the annotated relationship is 93.3% for the *PickPlace* task, 69.3% for the *Pouring* task, and 68.0% for the *Uprighting* task. While these annotations can be subjective and therefore are not an independent ground truth, the high agreement suggests that RAFAIL frequently identifies relationships that are consistent with the observed failure.

D. Ablations

We compare encoding and supervising individual relationships to encoding the whole scene by training and applying our relationship encoder to the whole scene point cloud. During the training, we use scene point clouds at the same task progress as positive and at different task progress as negative samples. While the relationship importance is not used in this setting, we still condition the OOD detection on the predicted task progress. Additionally, we investigate the

role of the predicted relationship importance by comparing to a version of RAFAIL that does not take the relationship importance into account during OOD detection which can easily be achieved by setting $\theta_{\text{imp}} = 0$ which removes the importance gate and considers all relationships to be important all the time. Additionally, we investigate using the predicted relationship importance to continuously weight the OOD scores using $f_t^k = \mathbb{I}[\hat{r}_t^k s_t^k > \theta_{\text{ood}}^k]$, rather than a gate. Table III shows the resulting Acc., TPR, and FPR across the three tasks. It indicates that both the relationship-wise encoding and the predicted relationship importance add to the final performance. Encoding the whole scene results in lower balanced accuracy across all tasks, suggesting that focusing the observation on individual relationships provides more suitable failure signals. Removing the influence of the predicted relationship importance by setting $\theta_{\text{imp}} = 0$ increases the TPR, but also the overall FPR from 11.6% to 69.8%, reducing the Acc. from 73.4% to 58.1%. Because the gate can only suppress detections, this trade-off can be task dependent: the $\theta_{\text{imp}} = 0$ variant maintains a consistently high TPR and achieves the highest Acc. for *Pouring*, where the relationship-wise OOD scores appear to separate successful and failed executions sufficiently well, but its high FPR makes it ineffective for *PickPlace* and *Uprighting*. This suggests that the predicted relationship importance provides a useful signal for suppressing OOD detection in currently less relevant relationships. Using the importance as a continuous weight rather than a gate results in a more conservative detector, with an improved overall FPR but also a reduced TPR of 37.8%. Its overall Acc. of 68.9% is however above that of the best evaluated baseline, indicating that both formulations can be effective while the gate provides the more favorable trade-off in the evaluated tasks.

V. CONCLUSION

We introduced RAFAIL, a relationship-aware framework for runtime failure detection that learns task-relevant object relationships from successful demonstrations. RAFAIL avoids runtime VLM inference; online semantic processing is limited to object tracking, relationship encoding, and RND evaluation. Across three real-world manipulation tasks, RAFAIL achieved 73.4% balanced accuracy, outperforming the strongest evaluated baseline while maintaining a comparatively low false-positive rate. The relationship-specific detections also frequently agreed with the relationships involved in the observed failures, supporting the usefulness of the relational representation. However, subtle failures caused by small pose deviations may remain within the distribution of successful relationship states and therefore go undetected. Future work should investigate finer-grained relational representations for detecting such errors. RAFAIL currently assumes that task-critical failure modes can be represented by pairwise relations among segmentable entities identified from successful demonstrations. Failures involving unmodeled entities, global scene properties, contact forces not visible geometrically, or segmentation/tracking breakdown may therefore remain undetected.

Task	Metric	RAFAIL			Scene
		gate	weight	$\theta_{\text{imp}} = 0$	
PICKPLACE	Acc. (%) $\uparrow$	66.7	63.3	44.9	23.2
	TPR (%) $\uparrow$	<u>33.3</u>	26.7	66.7	31.1
	FPR (%) $\downarrow$	0.0	0.0	76.9	84.6
POURING	Acc. (%) $\uparrow$	82.3	86.5	87.5	38.5
	TPR (%) $\uparrow$	<u>81.3</u>	72.9	91.7	77.1
	FPR (%) $\downarrow$	16.7	0.0	16.7	100.0
UPRIGHTING	Acc. (%) $\uparrow$	71.4	54.8	50.0	<u>54.8</u>
	TPR (%) $\uparrow$	<u>59.5</u>	9.5	100.0	9.5
	FPR (%) $\downarrow$	16.7	0.0	100.0	0.0
Overall	Acc. (%) $\uparrow$	73.4	68.9	58.1	43.6
	TPR (%) $\uparrow$	<u>58.5</u>	37.8	85.9	40.7
	FPR (%) $\downarrow$	11.6	0.0	69.8	53.5

TABLE III

ABLATIONS OF FAILURE DETECTION PERFORMANCE OF OUR PROPOSED RAFAIL (RAFAIL GATE) AND VARIANTS THAT USE RELATIONSHIP IMPORTANCE AS A CONTINUOUS WEIGHT (RAFAIL WEIGHT), IGNORE RELATIONSHIP IMPORTANCE (RAFAIL $\theta_{\text{imp}} = 0$), OR ENCODE THE WHOLE SCENE INSTEAD OF INDIVIDUAL RELATIONSHIPS (SCENE).

REFERENCES

- [1] R. Wolf, Y. Shi, S. Liu, and R. Rayyes, "Diffusion models for robotic manipulation: a survey," *Frontiers in Robotics and AI*, vol. 12, Sep. 2025.
- [2] E. Welte and R. Rayyes, "Interactive imitation learning for dexterous robotic manipulation: challenges and perspectives—a survey," *Frontiers in Robotics and AI*, vol. 12, Dec. 2025.
- [3] R. Römer, A. Kobras, L. Worbis, and A. Schoellig, "Failure prediction at runtime for generative robot policies," in *NeurIPS*, San Diego, CA, USA, December 2025.
- [4] Z. Lin et al., "Failsafe: Reasoning and recovery from failures in vision-language-action models," *arXiv preprint arXiv:2510.01642*, 2025.
- [5] Q. Gu et al., "Safe: Multitask failure detection for vision-language-action models," in *NeurIPS*, San Diego, CA, USA, December 2025.
- [6] J. Duan et al., "Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *ICLR*, Singapore, April 2025.
- [7] C. Grislain, H. Rahimi, O. Sigaud, and M. Chetouani, "I-failsense: Towards general robotic failure detection with vision-language models," *arXiv preprint arXiv:2509.16072*, 2026.
- [8] Q. Rolland, F. M. de Chamisso, and J.-B. Mouret, "Failure identification in imitation learning via statistical and semantic filtering," *arXiv preprint arXiv:2604.13788*, 2026.
- [9] C. Xu et al., "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," in *RSS*, Los Angeles, CA, USA, June 2025.
- [10] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *FTML*, vol. 16, no. 4, p. 494–591, March 2023.
- [11] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *ICLR*, New Orleans, USA, 2019.
- [12] H. Liu et al., "Multi-task interactive robot fleet learning with visual world models," in *CoRL*, Munich, Germany, 2024.
- [13] Qwen Team, "Qwen3.5: Towards native multimodal agents," February 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [14] Gemma Team, "Gemma 4 technical report," *arXiv preprint arXiv:2607.02770*, 2026.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, Las Vegas, NV, USA, July 2016.
- [16] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional lstm and other neural network architectures," *Neural Networks*, vol. 18, no. 5, pp. 602–610, 2005.
- [17] T. Ren et al., "Grounded sam: Assembling open-world models for diverse visual tasks," *arXiv preprint arXiv:2401.14159*, 2024.
- [18] N. Carion et al., "Sam 3: Segment anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.
- [19] Y. Wang et al., "Dynamic graph cnn for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, 2019.

- [20] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "Film: visual reasoning with a general conditioning layer," in *AAAI*, New Orleans, LA, USA, 2018.
- [21] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, "Putting the Object Back into Video Object Segmentation," in *CVPR*, Seattle, WA, USA, June 2024.
- [22] G. Yan et al., "ManiFlow: A general robot manipulation policy via consistency flow training," in *CoRL*, Seoul, Korea, September 2025.